Tractor Trace Format Description.

Files:

1. App (tractor_t.app[.gz])
2. Common (tractor_t.com.<rank>[.gz])
3. PTPD output (ptpd-tractor_t.log)
4. Maps (tractor_t.map.<rank>)
5. Main (tractor_t.mpi.<rank>[.gz])
6. Hardware (counters) file (tractor_t.hwc.<rank>[.gz])

   Transport API profile file (tractor_t.transport.<rank>[.gz]) - written only if the transport profiler is enabled

Format descriptions

Main MPI Format description:

1. All traced functions are divided into groups called classes. Now there are 9 MPI classes, one special record (MPI_Pcontrol) and 2 classes that exist only in tractor builds with the corresponding tracing enabled (OMPT, GPU):
   a. P2P (point to point (or peer to peer))
   b. PROBE (mpi_probe etc)
   c. REQUEST (MPI_WAIT/TEST etc)
   d. COLLECTIVE (MPI_BCAST etc)
   e. COMM or COMM_COLLECTIVE (communicator) MPI_COMM_CREATE etc.
   f. OSC (One Sided Communication) MPI_GET/PUT/ACCUMULATE etc
   g. FIO (File IO)
   h. BUFFERED_LOCAL (MPI_PACK/UNPACK)
   i. LOCAL (all other)

      PCONTROL (MPI_Pcontrol) - special record, see below

      OMPT (OpenMP tools events) - OMPT builds only

      GPU (OpenCL and Level Zero calls) - GPU builds only

Calls the MPI library issues internally (MPIC_*, MPID_* and similar, nested in a traced call) are written in the class of their kind: transfers inside collectives use the P2P layout, internal communicator creation/release uses the COMM layout.

2. A record of the main file is divided into 3 parts: fixed, class dependent and flexible (function/parameter dependent). The extra part (buffer addresses), if enabled, is written to the end of the hwc line.

3. Fixed part

   ids_section:thread_id:num_children:mpi_func_id:

   a. ids_section. id_columns ':' separated decimal signed values (id_columns is taken from the PEAT_RT_Number_Of_Events record of the common file, it is the same for all ranks of a trace). id_0 is the id of the top-level record: started from 1, incremental per rank. id_1… id_(n-1) form the path of a nested record (an MPI call the MPI library issued from inside another traced call): the index of the record among the children of its parent at each depth, -1 where unused. A top-level record has -1 in all columns but the first. The same ids open the corresponding line of the main (mpi) file; both files are line aligned.

   b. thread_id. Decimal, unsigned. OpenMP thread number, 0 if no OpenMP is involved. Every thread has its own tree of records; the ids are unique per rank, not per thread.

      num_children – decimal, signed. Number of records directly nested in this one: MPI calls the MPI library issued while this call was running (MPIC_* transfers of a collective, MPID_Request_Release, progress engine calls for another outstanding nonblocking operation and so on). The records of one thread form a pre-order tree: the num_children records following this record (each followed by its own subtree) are its children, in call order. A child's id at depth d is its position among its siblings, so the tree can also be rebuilt from the ids alone. The tractor keeps one storage per (thread, nesting level); the depth is limited by PEAT_TRACTOR_MAX_NESTING_LEVELS (default 4) and the deepest level reached defines id_columns.

   c. mpi_func_id – decimal, unsigned. Unique ID of MPI function[1].

4. Class dependent and flexible

   a. P2P and PROBE

      Class dependent part.

      :comm:peer:request:message:tag:transfer_bytes

1If T_PRINT_FUNC_NAMES is turned on this value is the string representing actual MPI call (like MPI_INIT).

1. comm – hexadecimal, unsigned. Global communicator id: 0 = MPI_COMM_NULL, 1 = MPI_COMM_SELF, 2 = MPI_COMM_WORLD, 3 and above are the ids of the global communicators report (PEAT_RT_Global_Communicators_Report in the app file). If the tractor runs without communicator resolution this is the raw MPI communicator handle.

2. peer – decimal, signed. Remote rank (number is in MPI_COMM_WORLD). Can contain wildcards (MPI_ANY_SOURCE) and MPI_PROC_NULL.

3. request – hexadecimal, unsigned. MPI Request. Valid only in case of non-blocking send/receive or persistent request. For MPI_IPROBE there is flag here. For other P2P and PROBE class function its value should be 0.

4. message– hexadecimal, unsigned. MPI Message. Valid only in case of send/receive/probe with message (MPI_MXXXX,MPI_ImXXXX, available in MPI3).

5. tag – decimal, signed. MPI TAG. Can contain wildcards (MPI_ANY_TAG).

6. transfer_bytes – decimal, signed. Number of transferred bytes[2].

There is no flexible part in case of P2P and PROBE class.

b. REQUEST

Class dependent part.

:input_request_num:triggered_request_num:flag:request_addr

1. input_request_num – decimal, signed. Number of input requests.

2. triggered_request_num – decimal, signed. number of request which complete during this call.

3. flag – decimal, signed. flag for MPI_TEST where applicable and -1 for any other case.

4. request_addr - hexadecimal, unsigned. Address of MPI_Requests(S).

Flexible part:

:v_0:v_1:…:v_n – hexadecimal, unsigned. Values of each request supplied to the function. N=input_request_num.

2 Meaningful only in case of actual sending or receiving. In probes its value is undefined. In case of persistent request (MPI_XXXX_Init) it shows actual value of sending or receiving bytes of corresponding MPI_STARTXXX call.

:t_{00}:t_{01}:t_{02}:t_{03}...:t_{m0}:t_{m1}:t_{m2}:t_{m3} – 4-tuple representing[3] (index(decimal, signed),tag(decimal, signed),source(decimal, signed),transferred_bytes(decimal, signed)) of each completed request. m=triggered_request_num. index=[0…input_request_num-1]

c. COLLECTIVE

Class dependent part.

:comm:root:request:send_bytes_num:recv_bytes_num

1. comm – hexadecimal, unsigned. Global communicator id: 0 = MPI_COMM_NULL, 1 = MPI_COMM_SELF, 2 = MPI_COMM_WORLD, 3 and above are the ids of the global communicators report (PEAT_RT_Global_Communicators_Report in the app file). If the tractor runs without communicator resolution this is the raw MPI communicator handle.

2. root – decimal, signed. Root process(in MPI_COMM_WORLD). -1 if no root process in this particular call.

3. send_bytes_num, recv_bytes_num - decimal, signed. Number of send and receive bytes fields.

4. Request - hexadecimal, unsigned. MPI_Request if applicable, 0 otherwise.

Flexible part:

:SB_0:SB_1:…:SB_p – decimal, signed. Bytes sent by this rank. p=send_bytes_num. Followed by :RB_0:RB_1:…:RB_q – decimal, signed. Bytes received by this rank. q=recv_bytes_num.

d. COMM

Class dependent part.

:comm:newcomm:remote_comm:request

comm, newcomm, remote_comm – hexadecimal, unsigned global communicator ids (same meaning as comm above), 0 if not applicable (not all three exist in all communicator functions). request – hexadecimal, unsigned. MPI_Request of a nonblocking communicator creation, 0 otherwise. No flexible part. The calls on the communicators the MPI library creates itself (mpi_internal_comm_create, mpi_internal_comm_free) have the same layout; such communicators are flagged as internal in the global communicators report.

e. OSC

3 Valid only for “receive” requests (f.i. initiated by mpi_irecv or mpi_recv_init). For send or null requests their value are undefined and should be ignored.

Class dependent part is pretty similar to P2P. All values have the same meanings except explicitly stated otherwise.

:comm:peer:request:window:size:transfer_bytes

1. size – decimal, signed. Size for window creation functions. Value should be ignored for all others.
2. window – hexadecimal, unsigned. MPI window handle.

This class has no flexible part.

f. FIO

Class dependent part.

Class dependent part (the same for all file functions, MPI_FILE_OPEN included):

:fh:comm:request:offset:transferred_bytes where

1. offset – decimal, signed. Offset if file if applicable. 0 otherwise.
2. fh – hexadecimal, unsigned. File handle.

   comm – hexadecimal, unsigned. Global communicator id (see comm above); it is the communicator of MPI_FILE_OPEN, 0 where not applicable.
3. request – hexadecimal, unsigned. MPIO_Request if applicable. 0 otherwise.
4. transferred_bytes – decimal, signed. Number of transferred bytes if applicable. 0 otherwise.

Legacy traces: MPI_FILE_OPEN had :comm:fh and the other file functions :offset:fh:request:transferred_bytes.

g. Buffered Local.

   :count:size

   1. count – decimal, signed.
   2. size – decimal, signed.

h. Local has no class dependent and flexible parts.

   PCONTROL

Class dependent part.

:ctl

ctl – decimal, signed. Level argument of MPI_Pcontrol. No flexible part.

OMPT (OpenMP tools events, OMPT builds only)

Class dependent part.

:kind:endpoint:task_data:parallel_data:code_addr_id

kind, endpoint – decimal, signed. OMPT event kind and scope endpoint. task_data, parallel_data – decimal, signed. OMPT task and parallel region data. code_addr_id – decimal. Id of the code address of the region (see PEAT_RT_User_Addresses in the common file). No flexible part.

GPU (OpenCL and Level Zero, GPU builds only)

Class dependent part.

:num_arguments:num_out_arguments:num_extra_arguments

All values are decimal, signed. Flexible part:

:in_0:...:in_n:out_0:...:out_m:extra_0:...:extra_k – hexadecimal, unsigned. Values of the input arguments (n=num_arguments), of the output arguments (m=num_out_arguments) and of the extra arguments (k=num_extra_arguments) of the call.

Legacy traces. Traces written before nesting support have two id columns primary_id:secondary_id followed by thread_id on both the main and the hwc lines, no num_children field, an additional num_colts field (number of nested records, all of depth one, listed after the call) in the REQUEST, COLLECTIVE and COMM class dependent parts, and TSC value [2] is MPI exit relative to the record start. Readers tell the two formats apart by the number of fields of the PEAT_RT_Number_Of_Events record (see Common).

Extra part:

Present only if the tractor was built with extra fields. Written at the end of the hwc line, after the memory profile section (not to the main file). Now only 1 extra part is defined:

1. Buffers addresses (only where applicable).
    a. Printed after all other sections of the hwc line.

        Number of buffers: FIO 1, BUFFERED_LOCAL 2, P2P and PROBE 1, OSC 3 (buf, rbuf, cbuf), REQUEST 2 (requests, statuses), COLLECTIVE 2 (send and receive buffers).

    b. Field separator is ‘:’.

    c. Every buffer is a tuple of ':' separated values: (virtual_address, size, physical_entry, flags) if VIRT2PHYS is enabled or (virtual_address, size) otherwise. virtual_address, physical_entry and flags are hexadecimal, size is decimal.

    d. If particular tuple is invalid all its values are zeros.

HWC Format description:

1. Each record is a single line. Field separator is ':'.
2. Each record corresponds a single mpi call with one exception: mpi_sendrecv and mpi_sendrecv_replace. These calls occupy 2 records.
3. The format is as follows:

   ids_section:TSCs_section:address_section:hardware_counters_section:memprofile_section

   a. ids_section. The same ids as on the corresponding mpi line (see above).

   b. thread_id is not present in this file (it is on the main file line only).

   c. TSC_Section: a ':' separated values. There are always 4 values. Their meaning is the same as in the transport records: [0] start of the record relative to the start of the previous record in the file (the previous line, whatever its nesting level; the very first record is relative to 0); [1] tool prolog: MPI entry minus record start; [2] tool epilog: record exit minus MPI exit; [3] duration: record exit minus record start. Pseudocode is as follows (tsc_val_base=0 at the very beginning):

```
tsc_start = read_val(id_columns + 0) + tsc_val_base;
tsc_val_base = tsc_start; // saved for the next record
mpi_in = tsc_start + read_val(id_columns + 1);
func_out = tsc_start + read_val(id_columns + 3);
mpi_out = func_out - read_val(id_columns + 2);
```

   MPI_Sendrecv and MPI_Sendrecv_replace (and their MPIC_ counterparts) occupy two lines with the same ids: the second line carries the receive part in the main file, its TSC values are all 0 (same start, no own timing) and it does not advance tsc_val_base.

   d. Address_section: a':' separated values. Number of values are taken from num_addrs read from app file (see below). The values here are the IDs of real addresses which may be read from app file. Pseudocode is as follows:

```
for (int addr_ind=0;addr_ind<num_addrs;addr_ind++) {
 addr_id=read_val(id_columns + 4 + addr_ind);
}
```

e. Hardware_counters_section: 2*num_hwcs ':' separated decimal values (num_hwcs is taken from the PEAT_RT_Var_Block_Info record of the app file; the section is empty if the tractor was built without hardware counters). The first num_hwcs values are the counters before the mpi call (i.e. in the user section preceding the mpi call), the next num_hwcs ones are the counters during the mpi call. With the default two counters (instruction_retired and clockticks) these are 4 values (2 pairs).

f. Memprofile_section (only if the tractor was built with the memory profiler): 8 ':' separated decimal values: vm_size, vm_RSS, ru_maxrss, mem_used_by_tractor at the record start, followed by the same four values at the record end.

Extra_section (only if the tractor was built with extra fields) is appended at the very end of the line, see 'Extra part' below.

MAP Format description:

Copy of /proc/pid/maps

Rest files format description

These files contain just records in flexible format (i.e. format depends on record number).

RECORD_ID:RECORD.

App Format description:

PEAT File Record Type Definitions.
An informative table to make correlation between id and theirs “high level” meaning. Can be used to translate messages between different version of tractor and pet/tv. (1 = PEAT_RT_Definitions) 1:id:name (e.g. PEAT_RT_Var_Block_Info). The ids are the values of the PEAT_Record_Type enumeration, names carry the PEAT_RT_ prefix. The full table is the first part of the app file.

PEAT_RT_Frame_Marker:decimal_value – currently not used.
PEAT_RT_Var_Block_Info:num_addrs:num_hwcs:ret_level_0:...:ret_level_(num_addrs-1) – number of address values per record of the hwc file, number of hardware counters (used to decode the hwc file) and the stack unwind (return) level of every address.
PEAT_RT_MPM_Master_Ranks:number:ranks – Master (first rank on each node) ranks. Number is decimal nonnegative value. ranks are separated by “:”.
PEAT_RT_MPM_Nodes_Freq:node_num:rank:freq0:freq1:init_delay:hostname. All values are decimal except hostname (string). Init_delay is now not used.
PEAT_RT_MPM_Ranks_And_PPN:hostname:frequency:ppn:id:ranks - mostly informational. Ranks separated by ‘,’.
PEAT_RT_MPM_All_Ranks_Exec_Time:nodes_num:3_tuples – three TSC time stamps per node gathered at MPI_Finalize; number of 3-tuples = nodes_num. Not used by the readers.

PEAT_RT_MPI_Function_Name_ID:id:MPI_FUNC – translation between function id (decimal) and function name (string)
PEAT_RT_Global_Communicators_Report:id:internal:size:ranks – one record per communicator. id – hexadecimal global communicator id (0, 1, 2 are MPI_COMM_NULL, MPI_COMM_SELF, MPI_COMM_WORLD and are not listed); this id is used in comm and p2p records and in the local communicators report. internal – decimal, 1 for a communicator created by the MPI library itself (mpi_internal_comm_create), 0 for a communicator created by the application. size – decimal, number of

ranks. ranks – ':' separated COMM_WORLD ranks of all members (for an inter communicator the ranks of both groups).
PEAT_RT_Sync_Point:size:<values> - Synchronization point. To be used if works without PTPD. Usually occurs twice (at the beginning and end of trace time).

Common Format description:

PEAT_RT_App_Start_Stop_Info:<start_tsc_val>:<stop_tsc_val>:<duration>:<tsc_init>
PEAT_RT_MPM_Rank_Time_Stamp:<app_start>:<app_start_parallel>:<app_end_parallel>:<app_end> – TSC time stamps of the rank (decimal).
PEAT_RT_CPU_Freq:0:<freq>:<freq>
PEAT_RT_Number_Of_Events:number_of_events:max_top_level_id:max_thread_id:id_columns:max_transport_id – all decimal. number_of_events counts the written records (nested ones included); id_columns is the number of id columns of the main and hwc files, max(2, deepest nesting level + 1), agreed among all ranks at MPI_Finalize; max_transport_id is the last id (first field) of the transport file, 0 if there is none.
PEAT_RT_Collector_Frame_Info: - currently reserved (not used)
PEAT_RT_User_Addresses:id:bin_address:binary_file:function:source_file:line
id, line – decimal
bin_addres – hexadecimal
All the rest – strings
PEAT_RT_Modules_Files_Definition:index:binary_file:base_address:load_address:end_address – index is decimal, addresses are hexadecimal, binary_file is a string.
PEAT_RT_Local_Communicators_Report:id:communicator:begin_time:end_time:size_local:size_remote:local_ranks[:remote_ranks] – one record per communicator of this rank. id – hexadecimal global communicator id (the id of the global communicators report in the app file), communicator – hexadecimal MPI handle, begin_time and end_time – decimal TSC (-1 if the communicator was not freed), sizes – decimal. local_ranks and remote_ranks are ',' separated COMM_WORLD ranks of the local and the remote group in the order of the local rank in the communicator; remote_ranks is absent for an intra communicator (size_remote is 0). The records of the user communicators are followed by the records of the MPI internal communicators.
PEAT_RT_HWLOC_Proc_Record:core_id and PEAT_RT_HWLOC_Node_Record – core the rank ran on and the node topology string.

PEAT_RT_MPI_Predefined:MPI_SUCCESS:MPI_UNDEFINED:MPI_COMM_WORLD: MPI_COMM_SELF: MPI_COMM_NULL: MPI_PROC_NULL:MPI_ANY_TAG:MPI_ANY_SOURCE:MPI_REQUEST_NULL:MPI_ROOT
Values are those of the traced MPI library (MPI_UNDEFINED, MPI_COMM_* and MPI_REQUEST_NULL are hexadecimal, the others, MPI_ROOT included, decimal); they are not the 0/1/2 communicator ids of the records.

PTPD Format description:

PEAT_RT_PTP_Mode:<dedicated_mode_in_use> (1 or 0)
PEAT_RT_PTP_Ranks:0:<number_of_ranks>:<rank_to_comm_world_mappings> - 'masters' rank mapping record

PEAT_RT_PTP_Nodes_Freq:<node_index>:<freq1>:<freq2>:<hostname> - frequency and hostname record
PEAT_RT_PTP_Main_Record:<destination_rank>:<begin_core>:<end_core>:t1:t2:t3:t4 - basic PTP record.

Transport API profile file (tractor_t.transport.<rank>[.gz]). Written only if the transport profiler is enabled; one line per transport call kind and thread. Field separator is ':'.
uid:thread_id:function:tsc0:tsc1:tsc2:tsc3:total_clocks_between_calls:num_calls

uid – decimal, incremental per rank (the last one is max_transport_id of PEAT_RT_Number_Of_Events). thread_id – decimal. function – function id (decimal) or name. tsc0 – start of the record relative to the start of the previous line; tsc1, tsc2, tsc3 – absolute TSC time stamps of the record: entry into the transport call, exit from it and end of the record. total_clocks_between_calls and num_calls – decimal, accumulated for the function over the whole run.

Bind files (written by mat)

Bind files (tractor_t.bind.<rank>.out[.gz]) are not written by the tractor, they are produced by mat and pet from a trace and read by pet and qTV. They refer to trace records by rank and full id path, so their width follows id_columns of the trace. Field separator is ':', RECORD_ID first.
PEAT_RT_Bind_Peer:function:rank:ids:function:rank:ids:size – a send record bound to its receive record (ids = id_columns values each).
PEAT_RT_Bind_Completion:function:ids:function:ids and PEAT_RT_Bind_Initiation:function:ids:function:ids – a nonblocking or persistent record and the completing (starting) call of the same rank.
PEAT_RT_Bind_Collective:function:rank:ids:rank:ids:… – the member records of one collective call, one rank:ids pair per participating rank.